Módulo
Nome: **[BE-JV-013] Fundamentos de segurança para aplicações**
Horários
Segunda | Quarta | Sexta : 15:00 – 18:00 Período: 28/10/2026 a 18/11/2026

Objetivo

O objetivo do módulo é que os alunos compreendam os fundamentos da segurança para aplicações. Serão apresentados credenciais, autenticação e autorização (OAuth, SSO e OpenID Connect), autenticação de APIs (padrão JWT), tratamento e isolamento de exceções, proteção de entrypoints, sanitização e defesa contra injection e o Spring Security.

Ao final, espera-se que os alunos sejam capazes de implementar uma camada de autenticação em uma aplicação REST utilizando Spring Security, com autenticação básica ou com JWT.

Conteúdo

- O que é segurança de aplicativos web e por que protegê-los
- Tipos de ataque: Cross-site Scripting (XSS), Cross-site Request Forgery (CSRF) e Denial-of-Service (DoS)
- Noções básicas de criptografia e HTTPS
- Cookies
- Autenticação baseada em sessão e autenticação baseada em token
- Json Web Token (JWT)
- OAuth 2.0
- Single Sign-On (SSO)
- OpenID Connect
- Situações reais de vulnerabilidade (SQL Injection e dependências vulneráveis)
- Tratamento de exceções
- Introdução ao Spring Security
- Implementação de autenticação básica
- Implementação de autenticação com JWT

Aulas

- Aula 01 - Introdução à segurança de aplicações: XSS, CSRF e DoS

- Aula 02 - Noções básicas de HTTPS e Cookies
- Aula 03 - Json Web Token (JWT)
- Aula 04 - OAuth 2.0
- Aula 05 - Single Sign-On (SSO)
- Aula 06 - OpenID Connect
- Aula 07 - Situações reais de vulnerabilidade e Tratamento de exceções
- Aula 08 - Spring Security: autenticação básica e JWT
- Aula 09 - Desenvolvimento do Projeto e Encerramento

Justificativa

Aplicações web e APIs estão expostas continuamente a tentativas de invasão, vazamento de dados e indisponibilidade. Falhas de segurança geram prejuízo financeiro, perda de confiança dos usuários e impactos legais para as empresas, e a maior parte delas nasce de decisões tomadas durante o desenvolvimento: entradas não tratadas, credenciais mal protegidas, dependências desatualizadas e mensagens de erro que revelam mais do que deveriam. Por isso, segurança precisa ser tratada pela pessoa desenvolvedora desde as primeiras linhas de código, e não apenas como uma etapa posterior ao desenvolvimento.

Este módulo justifica-se pela necessidade de apresentar os fundamentos que sustentam a segurança de aplicações: os principais tipos de ataque, o papel da criptografia e do HTTPS, os mecanismos de autenticação e autorização (sessão, token, JWT, OAuth, SSO e OpenID Connect) e o tratamento seguro de exceções. Esses conceitos são aplicados com o Spring Security, framework padrão para proteger aplicações Spring, e consolidados em um projeto no qual os alunos implementam uma camada de autenticação em uma API REST. Ao final, espera-se que os alunos reconheçam vulnerabilidades comuns e saibam aplicar as defesas adequadas em suas aplicações.

Aulas e descrição

Aula 01 - Data 28/10 - Introdução à segurança de aplicações: XSS, CSRF e DoS
Tópicos ● Apresentação do instrutor ● Apresentação dos alunos ● Apresentação do plano de aula e do projeto do módulo ● Por que precisamos proteger aplicativos? ● Ataques Cross-site Scripting (XSS), Cross-site Request Forgery (CSRF) e Denial-of-Service (DoS) **Objetivo** Iniciar com a apresentação do instrutor e dos alunos, criando um ambiente de interação, e apresentar o plano de aula e o projeto do módulo. Em seguida, compreender por que as aplicações precisam ser protegidas e conhecer os três ataques mais comuns a aplicações

web (XSS, CSRF e DoS), como funcionam e como preveni-los.

Problema Gerador

Introdução à segurança da informação: o que acontece quando uma aplicação não é protegida?

Assuntos a ser apresentados:

- O que é segurança de aplicativos web?
- Por que precisamos proteger aplicativos?
  - Vantagens de proteger sua aplicação
  - Casos recentes de ataques (Poly Network, Liquid e Renner, em 2021)
  - Vulnerabilidades dos sites
- Cross-site Scripting (XSS)
  - O que é Cross-site Scripting?
  - Tipos de ataque: Stored XSS e Reflected XSS
  - Exemplo de script malicioso
  - Como prevenir: validação e limpeza da entrada do usuário, Content Security Policy (CSP) e cabeçalhos do Spring Security
- Cross-site Request Forgery (CSRF)
  - O que é e como funciona (sessão baseada em cookies e parâmetros de requisição previsíveis)
  - CSRF com método GET
  - CSRF com método POST
  - Como prevenir: token CSRF e cookies SameSite
- Denial-of-Service (DoS)
  - O que é Denial-of-Service?
  - Tipos de ataque: Flood Attack e Crash Attack
  - Diferença entre ataque DoS e DDoS
  - Como parar um ataque: Black Hole Routing e Rate Limit (exemplo com resilience4j)

Prática:

- Dinâmica em sala "O valor da segurança": em grupos, listar o que uma empresa pode perder quando sua aplicação é invadida (dados, dinheiro, reputação) e o que seria preciso para evitar.
- Estudo de caso: como prevenir CSRF? Entendendo um DoS simples.
- Executar um ataque Reflected XSS no Google XSS Game (xss-game.appspot.com), criado pelo Google em 2014.
- Lista de exercícios de múltipla escolha sobre XSS.
- Lista de exercícios de múltipla escolha sobre CSRF.
- Lista de exercícios de múltipla escolha sobre DoS.

Atividade para casa

Ler sobre Segurança em aplicação web (MDN Web Docs - Website Security) e a página da OWASP sobre XSS (https://owasp.org/www-community/attacks/xss/).

Pesquisar como um script malicioso pode atacar o usuário.

Pesquisar sobre grandes ataques DoS/DDoS que já aconteceram.

Aula 02 - Data 30/10 - Noções básicas de HTTPS e Cookies

Tópicos

- O que é criptografia?
- O que é HTTPS e como funciona?
- Como HTTPS pode ser atacado?
- Tornando HTTPS seguro
- O que são cookies?

Objetivo

Compreender os conceitos básicos de criptografia e como o HTTPS os utiliza para proteger a comunicação entre cliente e servidor, reconhecer os ataques possíveis ao HTTPS e entender o papel dos cookies e de seus atributos de segurança.

Problema Gerador

Noções básicas de HTTPS: como garantir que ninguém leia ou altere o que trafega entre o navegador e o servidor?

Assuntos a ser apresentados:

- O que é criptografia?
    - Criptografia simétrica e assimétrica (chave pública e chave privada)
    - Hash e integridade dos dados
- O que é HTTPS e como funciona?
    - Diferença entre HTTP e HTTPS
    - O que o HTTPS garante: confidencialidade, autenticidade e integridade
    - Certificados SSL/TLS, autoridade certificadora e visão geral do handshake
- Como HTTPS pode ser atacado?
    - Fraquezas do protocolo ou da codificação
    - Dispositivo do usuário comprometido por phishing
    - Certificado falso se passando por um certificado verdadeiro
- Tornando HTTPS seguro
    - Certificado emitido por uma autoridade de certificação e dentro da validade
    - Ter HTTPS na URL não significa que o site é seguro
    - Benefício adicional: melhor classificação em pesquisas
- O que são cookies?
    - Composição de um cookie: nome, valor e atributos
    - Tipos: primários, de terceiros, de sessão e permanentes
    - Atributos de segurança: Secure, HttpOnly e SameSite
    - Lendo um cookie com Spring (@CookieValue)

Prática:

- Estudo de caso "Entendendo HTTPS": inspecionar o certificado de um site pelo

cadeado do navegador e identificar emissor e validade.
- Usar as ferramentas do desenvolvedor do navegador para observar os cookies de um site e seus atributos.
- Criar um endpoint Spring que leia um cookie com @CookieValue, com e sem valor padrão (defaultValue).
- Lista de exercícios de múltipla escolha sobre HTTPS.

Atividade para casa

Ler "Proteja seu site com HTTPS" (Google Search Central):
https://developers.google.com/search/docs/advanced/security/https?hl=pt-br

Pesquisar a diferença entre SSL e TLS e quais versões são consideradas seguras atualmente.

Pesquisar a diferença entre criptografia simétrica e assimétrica e dar um exemplo de uso de cada uma.

Aula 03 - Data 04/11 - Json Web Token (JWT)

Tópicos

- Autenticação baseada em sessão
- Autenticação baseada em token
- O que é JWT - Json Web Token?
- Validação JWT
- JWT Stolen
- Gerenciamento de chave de criptografia

Objetivo

Compreender as diferenças entre autenticação baseada em sessão e em token, a estrutura de um JWT, como ele é validado e quais são os riscos e os cuidados em caso de roubo do token e de exposição da chave de assinatura.

Problema Gerador

Json Web Token (JWT): como autenticar usuários em uma API sem guardar sessão no servidor?

Assuntos a ser apresentados:

- Autenticação baseada em sessão/cookies
  - Fluxo básico de autenticação baseada em sessão
  - Problemas: sistemas distribuídos, desempenho e fraude (CSRF)
- Autenticação baseada em token
  - Fluxo básico: login, emissão do token e envio do token a cada requisição
  - Benefícios: escalabilidade (stateless), segurança e controle de acesso
  - Tipos de tokens: token de acesso e refresh token
- O que é JWT?

    - Estrutura: header (alg e typ), payload e signature
    - Tipos: assinado (JWS), criptografado (JWE), assinado e criptografado e inseguro
    - Base64 não é criptografia: o conteúdo do payload é legível por quem tem o token
- Validação JWT
    - Verificação da assinatura e do tempo de expiração
- JWT Stolen (roubado)
    - Formas de roubo de um token: XSS, tráfego sem HTTPS e armazenamento inseguro no cliente
    - Como limitar o dano: tempo de expiração curto, HTTPS e refresh token
- Gerenciamento de chave de criptografia
    - Chave secreta (HMAC) x par de chaves pública/privada (RSA)
    - Proteção e rotação da chave; nunca versionar a chave no repositório

Prática:

- Estudo de caso "Estrutura JWT": decodificar um token no jwt.io e identificar header, payload e assinatura.
- Alterar o payload de um token e observar a falha na validação da assinatura.
- Discussão: o que um atacante consegue fazer com um JWT roubado e como limitar o dano?
- Lista de exercícios de múltipla escolha sobre JWT.

Atividade para casa

Ler a introdução ao JWT: https://jwt.io/introduction

Ler o artigo da Okta "What happens if your JWT is stolen?": https://developer.okta.com/blog/2018/06/20/what-happens-if-your-jwt-is-stolen

Pesquisar o que são os claims registrados de um JWT (iss, sub, aud, exp, iat) e para que serve cada um.

Aula 04 - Data 06/11 - OAuth 2.0

Tópicos

- Introdução ao OAuth
- Terminologias OAuth
- Grant Type código de autorização
- Implícito Grant Type
- Grant Type (concessão de credenciais) de credenciais do cliente
- Grant Type (concessão de credenciais) de credenciais do proprietário do recurso
- Atualização de token Grant

Objetivo

Compreender o OAuth 2.0 como protocolo de autorização, identificar seus papéis e

terminologias e escolher o grant type adequado para cada cenário.

Problema Gerador

OAuth: como permitir que um aplicativo acesse seus dados em outro serviço sem que você entregue sua senha a ele?

Assuntos a ser apresentados:

- O que é OAuth?
  - Protocolo de autorização: acesso limitado a contas de usuários sem compartilhar usuário e senha
  - Exemplo: login social (social sign-in)
- O que é OAuth 2.0?
- Terminologias OAuth
  - Proprietário do recurso, cliente, servidor de recursos e servidor de autorização
  - Concessão de autorização, código de autorização, token de acesso e escopo
- Grant types (concessões)
  - Código de autorização (Authorization Code)
  - Implícito (Implicit)
  - Credenciais do cliente (Client Credentials)
  - Credenciais do proprietário do recurso (Resource Owner Password Credentials)
  - Atualização de token (Refresh Token)
- Qual grant type usar hoje: os grants implícito e de credenciais do proprietário são considerados legados; a recomendação atual é código de autorização com PKCE
- Por que não usar OAuth para autenticação? OAuth autoriza; a autenticação é feita pelo OpenID Connect

Prática:

- Estudo de caso: mapear o fluxo "Entrar com Google/GitHub" identificando proprietário do recurso, cliente, servidor de autorização e servidor de recursos.
- Dinâmica: dado um cenário (aplicação web com backend, SPA, aplicativo mobile, integração servidor a servidor), escolher e justificar o grant type.
- Lista de exercícios de múltipla escolha sobre OAuth.

Atividade para casa

Ler a introdução ao OAuth 2.0: https://oauth.net/2/

Pesquisar o que é PKCE e qual problema ele resolve.

Escolher um aplicativo que você usa com login social e listar quais permissões (escopos) ele solicita.

Aula 05 - Data 09/11 - Single Sign-On (SSO)

Tópicos

- O que é Single Sign-On?
- Integrações SSO
- Identidade federada
- Fluxo de trabalho
- Arquitetura
- Tipos de protocolo (SAML 2.0, OpenID Connect)

Objetivo

Compreender o que é Single Sign-On, o conceito de identidade federada, o fluxo e a arquitetura de uma solução de SSO e os protocolos utilizados.

Problema Gerador

Single Sign-On: como entrar uma única vez e acessar vários sistemas?

Assuntos a ser apresentados:

- O que é Single Sign-On?
    - Login único: um conjunto de credenciais para acessar vários aplicativos e serviços
    - Exemplo: login na conta Google dando acesso ao Gmail e ao YouTube
- Identidade federada
    - Compartilhamento de atributos de identidade entre sistemas confiáveis, mas autônomos
- Arquitetura
    - Provedor de identidade, aplicações (provedores de serviço) e relação de confiança
- Fluxo de trabalho
    - O usuário solicita um recurso, a aplicação redireciona para autenticação, o usuário informa as credenciais, o SSO valida e devolve a resposta e a aplicação conclui o login
- Tipos de protocolo
    - SAML 2.0, OAuth e OpenID Connect
- Integrações SSO
    - Exemplos de provedores: Google, Microsoft Entra ID, Keycloak e Okta
- Vantagens e riscos
    - Menos senhas e gestão centralizada de acessos; o provedor de identidade passa a ser um ponto único de falha e um alvo valioso

Prática:

- Debate sobre casos de uso: onde o SSO aparece no dia a dia (conta Google, sistemas corporativos, serviços do governo) e quais as vantagens e riscos de cada um.
- Em grupos, desenhar o fluxo de SSO entre duas aplicações e um provedor de identidade.
- Lista de exercícios de múltipla escolha sobre Single Sign-On.

Atividade para casa

Ler "O que é Single Sign-On" (GeeksForGeeks):
https://www.geeksforgeeks.org/introduction-of-single-sign-on-sso/

Pesquisar a diferença entre SAML 2.0 e OpenID Connect (formato das mensagens e cenários de uso).

Escolher um sistema que você usa com SSO e descrever quem é o provedor de identidade e quem são as aplicações.

Aula 06 - Data 11/11 - OpenID Connect

Tópicos

- Introdução ao OpenID Connect
- Terminologias OpenID Connect
- Fluxo de autenticação
- Fluxo de código de autorização
- Fluxo implícito de autenticação

Objetivo

Compreender o OpenID Connect como camada de autenticação sobre o OAuth 2.0, identificar seus participantes e comparar o fluxo de código de autorização com o fluxo implícito.

Problema Gerador

OpenID Connect: como um aplicativo descobre quem é o usuário sem receber a senha dele?

Assuntos a ser apresentados:

- O que é OpenID Connect?
  - Extensão do OAuth: camada fina que adiciona suporte à autenticação
  - OAuth autoriza; OpenID Connect autentica
  - Versões: OpenID 1.0, OpenID 2.0 e OpenID Connect
- Terminologias OpenID Connect
  - ID Token (JWT com a identidade do usuário), token de acesso e escopo openid
- Participantes do OpenID Connect
  - Usuário final, cliente, provedor de recursos e servidor de autorização
- Fluxo de código de autorização
  - Redirecionamento para o provedor de identidade, login, retorno com código de autorização, troca do código por token e acesso ao recurso
- Fluxo implícito
  - O token é devolvido diretamente no redirecionamento; perde aspectos de segurança e hoje é desaconselhado
- Relação com JWT, OAuth e SSO

Prática:

- Estudo de caso: analisar o login por rede social de um e-commerce e identificar os participantes do OpenID Connect.
- Decodificar um ID Token de exemplo no jwt.io e identificar os claims de identidade (iss, sub, aud, name e email).
- Montar um quadro comparativo entre OAuth, OpenID Connect, SSO e SAML.
- Lista de exercícios de múltipla escolha sobre OpenID Connect.

Atividade para casa

Ler a página do OpenID Connect: https://openid.net/connect/

Pesquisar a diferença entre ID Token e token de acesso.

Revisar as aulas 3 a 6 e escrever, com suas palavras, em até 10 linhas, a relação entre JWT, OAuth, SSO e OpenID Connect.

Aula 07 - Data 13/11 - Situações reais de vulnerabilidade e Tratamento de exceções

Tópicos

- Situações reais de vulnerabilidade: SQL Injection e dependências vulneráveis
- Entendendo exceções
- Exceções baseadas em Controller
- Manipulação global de exceções

Objetivo

Reconhecer situações reais de vulnerabilidade (SQL Injection e dependências vulneráveis) e tratar exceções de forma segura em uma API Spring, sem expor informações sensíveis, com exceções próprias e tratamento global.

Problema Gerador

Tratamento de exceções: o que a sua API revela quando algo dá errado?

Assuntos a ser apresentados:

- Situações reais de vulnerabilidade
  - OWASP Top 10
  - SQL Injection: exemplo com concatenação de texto e prevenção com instruções preparadas (consultas parametrizadas)
  - Dependências com vulnerabilidades: o caso Log4j (Log4Shell), como manter as dependências atualizadas e ferramentas que identificam pacotes vulneráveis
- Entendendo exceções
  - Exceções são eventos inesperados e, em Java, são objetos; uma exceção não capturada encerra o programa
  - Tratamento seguro: detectar todas as exceções, não deixar o sistema em estado inseguro (não conceder acesso em caso de erro) e não divulgar informações sensíveis na resposta

- Implementação de tratamento de exceções com try-catch-finally
- Implementação de exceções próprias para a aplicação
- Exceções baseadas em Controller (@ExceptionHandler e ResponseStatusException)
- Manipulação global de exceções (@ControllerAdvice/@RestControllerAdvice), com corpo de erro padronizado e status HTTP adequado

Prática:

- Demonstração: busca de alunos vulnerável a SQL Injection (concatenação) e a versão corrigida com consulta parametrizada.
- Implementar tratamento de exceção com try-catch e uma exceção própria.
- Implementar tratamento de exceção global na API de alunos.
- Lista de exercícios para implementação de tratamento de exceção com bloco try-catch.
- Lista de exercícios para implementação de tratamento de exceção global.

Atividade para casa

Na API de alunos (ou na API que será usada no projeto), criar uma exceção própria e um tratamento global que retorne um JSON padronizado, sem stack trace.

Ler sobre o OWASP Top 10 (https://owasp.org/www-project-top-ten/) e escolher uma categoria para explicar em 3 linhas.

Ler "Error Handling for REST with Spring" (Baeldung): https://www.baeldung.com/exception-handling-for-rest-with-spring

Aula 08 - Data 16/11 - Spring Security: autenticação básica e JWT

Tópicos

- Introdução ao Spring Security
- Implementação de autenticação básica
- Implementação de autenticação com JWT
- Spring Security além da autenticação

Objetivo

Compreender os conceitos do Spring Security e implementar a proteção dos endpoints de uma API REST, primeiro com autenticação básica e depois com autenticação via JWT.

Problema Gerador

Spring Security: como implementar autenticação com JWT na API?

Assuntos a ser apresentados:

- Introdução ao Spring Security
    - Estrutura de autenticação e controle de acesso, padrão de fato para proteger aplicações Spring

    - Características: autenticação e autorização, proteção contra session fixation, clickjacking e CSRF, integração com Spring Web MVC
- Implementação de autenticação básica
    - Dependência spring-boot-starter-security
    - Comportamento padrão: formulário de login e senha gerada no log
    - Definição de usuário e senha no application.yml
    - Liberação e proteção de rotas
- Implementação de autenticação com JWT
    - Fluxo: o cliente envia as credenciais, recebe um token JWT e o envia nas próximas requisições
    - Entidade User, UserRepository e senha armazenada com BCrypt (PasswordEncoder)
    - Endpoint de criação de usuários (POST /user)
    - Endpoint de autenticação (POST /authenticate), que retorna o token JWT
    - Filtro que valida o token a cada requisição e configuração de segurança da aplicação
- Spring Security além da autenticação
    - Cookies, proteção contra XSS (cabeçalhos) e CSRF (ativa por padrão)
    - Quando é aceitável desabilitar o CSRF: APIs stateless que não usam cookies

Observação para o instrutor:

- O material de apoio usa WebSecurityConfigurerAdapter, removida no Spring Security 6. Na demonstração, utilizar a configuração por bean SecurityFilterChain.
- O projeto pede o token no header "access_token"; o padrão de mercado é "Authorization: Bearer <token>". Apresentar o padrão e combinar com a turma qual será usado no projeto.

Prática:

- Estudo de caso: como implementar autenticação com JWT na API?
- Adicionar o Spring Security à API de alunos, observar a senha gerada, configurar usuário e senha e proteger rotas específicas.
- Implementar o cadastro de usuários e o endpoint de autenticação com JWT e testar o fluxo completo no Postman ou via curl (cadastrar, autenticar e acessar um endpoint protegido com e sem token).
- Exercícios de fixação (JWT).

Atividade para casa

Concluir em casa o que não foi finalizado em aula e começar o projeto do módulo: escolher a API (nova ou existente), modelar a entidade User e implementar o POST /user.

Ler a documentação do Spring Security: https://docs.spring.io/spring-security/reference/getting-spring-security.html

Pesquisar por que senhas devem ser armazenadas com hash e salt (BCrypt) e nunca em texto puro.

Aula 09 - Data 18/11 - Desenvolvimento do Projeto e Encerramento

Tópicos

- Desenvolvimento do projeto do módulo
- Apresentação dos projetos
- Devolutiva da avaliação por rubrica / auto-avaliação

Objetivo

Dedicar a aula ao desenvolvimento e à entrega do projeto do módulo, dando suporte aos alunos, e finalizar com a apresentação dos projetos e a devolutiva da avaliação por rubrica.

Problema Gerador

Desenvolvimento do Projeto: implementar uma camada de autenticação em uma aplicação REST.

Requisitos do projeto:

- Aplicação REST nova ou já existente, protegida com Spring Security simples ou com JWT.
- No caso de JWT, criar o endpoint de criação de usuários POST /user e persistir os dados em um banco de dados de preferência do aluno.
- No caso de JWT, criar o endpoint de autenticação POST /authenticate, que recebe os dados do usuário e retorna um token JWT.
- Com o token em mãos, acessar os demais endpoints da aplicação enviando o token no header.
- Endpoints sem token ou com token inválido devem ser rejeitados com o status HTTP adequado.

Prática:

- Desenvolvimento do projeto em aula, com apoio do instrutor aos alunos e grupos.
- Apresentação dos projetos: demonstrar o fluxo de criar um usuário, autenticar e acessar um endpoint protegido com e sem token.
- Devolutiva da avaliação por rubrica e auto-avaliação.

Links para visitar

- https://spring.io/projects/spring-security
- https://docs.spring.io/spring-security/reference/getting-spring-security.html
- https://jwt.io/introduction
- https://oauth.net/2/
- https://openid.net/connect/
- https://owasp.org/www-community/attacks/xss/

- https://owasp.org/www-community/attacks/csrf
- https://owasp.org/www-project-top-ten/
- https://xss-game.appspot.com/
- https://developer.mozilla.org/en-US/docs/Learn/Server-side/First_steps/Website_security
- https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
- https://developers.google.com/search/docs/advanced/security/https?hl=pt-br
- https://resilience4j.readme.io/docs/ratelimiter
- https://www.baeldung.com/exception-handling-for-rest-with-spring
- https://www.geeksforgeeks.org/session-vs-token-based-authentication/

Bibliografia

- Spring Security in Action (Laurențiu Spilcă)
- OAuth 2 in Action (Justin Richer e Antonio Sanso)
- Web Application Security (Andrew Hoffman)
- Effective Java (Joshua Bloch)
- Documentação oficial do Spring Security: https://docs.spring.io/spring-security/reference/